

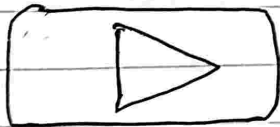
UNIT - 02

LECTURE - 01

COMPUTER ORGANIZATION & ARCHITECTURE

Topics:-

- ① Look Ahead Carry Adders
- ② Addition and Subtraction
- ③ Hardware Implementation
- ④ Addition and Subtraction operation



SUBSCRIBE = @MULTIATOMS
&
@MULTIATOMS +

LOOK AHEAD CARRY ADDER

Q Explain in detail the principle of carry look ahead adders and design 4-bit CLA adder {AKTU 2023-2024}
or

Q Describe the derivation procedure of look ahead carry adder by an example with the help of block diagram. {AKTU 2022-2023}
or

Q

Sol Q The CLA adder is a type of digital circuit used to add binary numbers quickly by calculating carry bits faster.

① It reduces the propagation delay, which occurs during addition.

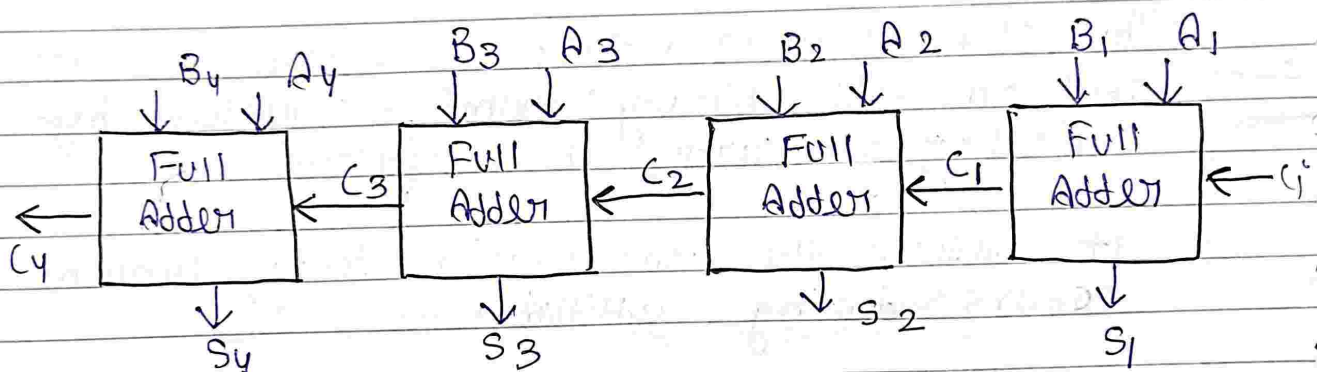
② It uses more complex hardware circuitry

③ It is designed by transforming the ripple-carry adder circuit, such that the carry logic of the adder is changed into two-level logic

Problem with Ripple Carry Adders :-

- ① In a regular adder, called a ripple carry adder, the carry bit from each addition must "ripple" to the next stage.
- ② This makes it slow for longer numbers, as each bit has to wait for the carry from the previous bit.

For ex:- 4-bit - Ripple Carry - Adder



$$\begin{array}{r} \quad \quad \quad \overset{C_3}{\curvearrowright} \overset{C_2}{\curvearrowright} \overset{C_1}{\curvearrowright} \\ \quad \quad \quad A_4 A_3 A_2 A_1 \\ + \quad B_4 B_3 B_2 B_1 \\ \hline C_4 S_4 S_3 S_2 S_1 \end{array}$$

Sol: The CLA adder solves by calculating every bits in parallel, instead of waiting for each stage

⇒ Truth table of Look Ahead adder

A	B	C_i	Carry
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

$A \cdot B \rightarrow$ Generation stage -

$[A \oplus B] \cdot C \rightarrow$ Carry Propagation stage

$$C_{i+1} = A_i B_i + C_i [A_i \oplus B_i]$$

Carry
Generated

Carry Propagated

$$G_i = A_i \cdot B_i$$

$$P_i = A_i \oplus B_i$$

○ For $i = 0, 1, 2, 3$, the output carry

$$C_1 = C_0 (A_0 \oplus B_0) + A_0 B_0$$

$$C_2 = C_1 (A_1 \oplus B_1) + A_1 B_1$$

$$C_3 = C_2 (A_2 \oplus B_2) + A_2 B_2$$

$$C_4 = C_3 (A_3 \oplus B_3) + A_3 B_3$$

where

G_i : Carry generator (which produce the carry when both A_i, B_i are one)

P_i : Carry propagator (it is associate with the propagation of carry from (i) to $(i+1)$)

$$\text{Sum output } = S_i = A_i \oplus B_i \oplus C_i = P_i \oplus C_i$$

$$\text{Carry output} = C_{i+1} = \cancel{G_i + P_i C_i} = A_i \cdot B_i + C_i (A_i \oplus B_i)$$

For $i = 0, 1, 2, 3$

Therefore the carry bits C_1, C_2, C_3 and C_4 :

$$C_1 = C_0 \cdot P_0 + G_0$$

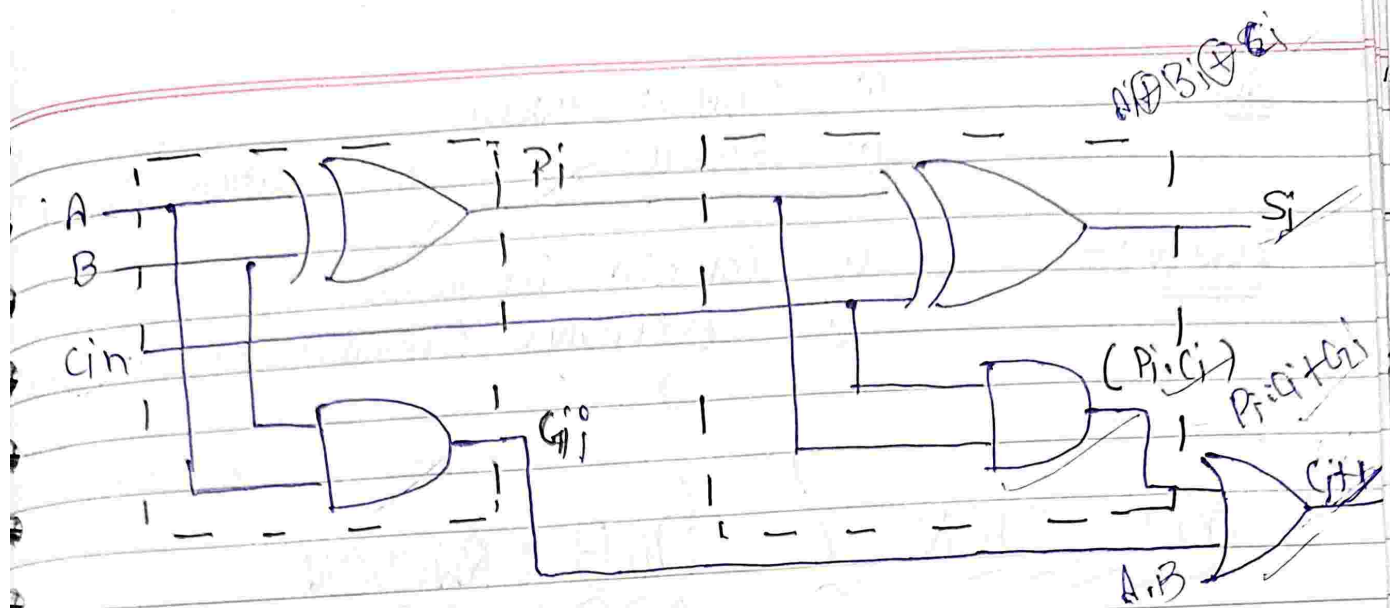
$$C_2 = C_1 \cdot P_1 + G_1 = (C_0 \cdot P_0 + G_0) \cdot P_1 + G_1$$

$$C_3 = C_2 \cdot P_2 + G_2 = [(C_0 \cdot P_0 + G_0) \cdot P_1 + G_1] \cdot P_2 + G_2$$

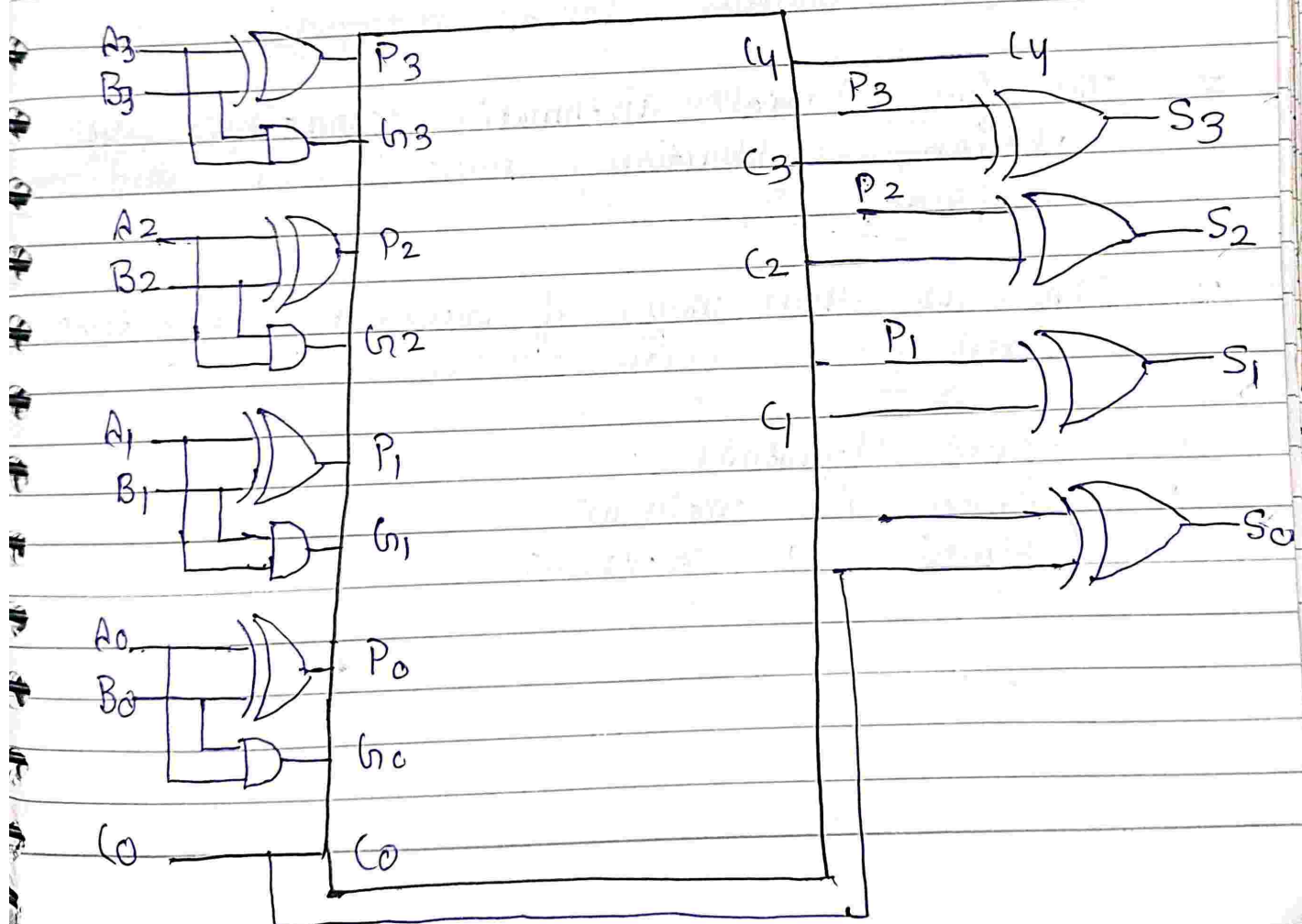
$$C_4 = C_3 \cdot P_3 + G_3 = C_0 \cdot P_0 \cdot P_1 \cdot P_2 \cdot P_3 + P_3 \cdot P_2 \cdot P_1 \cdot G_0 + P_3 \cdot P_2 \cdot G_1 + G_2 \cdot P_3 + G_3$$

Equations show that the carry-in of any stage full adder depends only on :-

- ① Bits being added in the previous stages
- ② Carry bits which was provided in the beginning



Look Ahead Carry Adder Circuit Diagram



Adv :-

- ① Fastest adder
- ② Reduces the propagation delay

Disadv :-

- ① Complex hardware
- ② Expensive (involve complex hardware)

ADDITION & SUBTRACTION

=> Arithmetic instructions manipulate data to produce solution for computational problem

=> The four basic arithmetic operations are addition, subtraction, multiplication and division

∴ There are three ways of representing negative fixed point binary numbers:

- Signed Magnitude
- Signed 1's complement
- Signed 2's complement

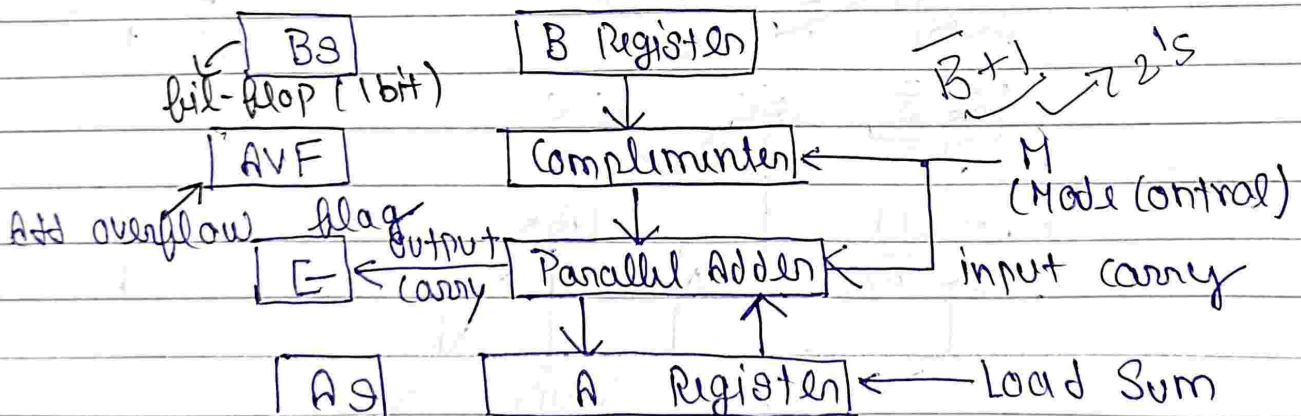
⇒ When two signed numbers A and B are added and subtracted, we find 8 different conditions

+5 1⁶

Subtract

Operation	ADD	$A > B$	$A < B$	$A = B$
$(+A) + (+B)$	$+(A+B)$	$+(A-B)$	$-(B-A)$	$+(A-B)$
$(+A) + (-B)$		$-(A-B)$	$+(B-A)$	$+(A-B)$
$(-A) + (+B)$				
$(-A) + (-B)$	$-(A+B)$			
$(+A) - (+B)$		$+(A-B)$	$-(B-A)$	$+(A-B)$
$(+A) - (-B)$	$+(A+B)$			
$(-A) - (+B)$	$-(A+B)$			
$(-A) - (-B)$		$-(A-B)$	$+(B-A)$	$+(A-B)$

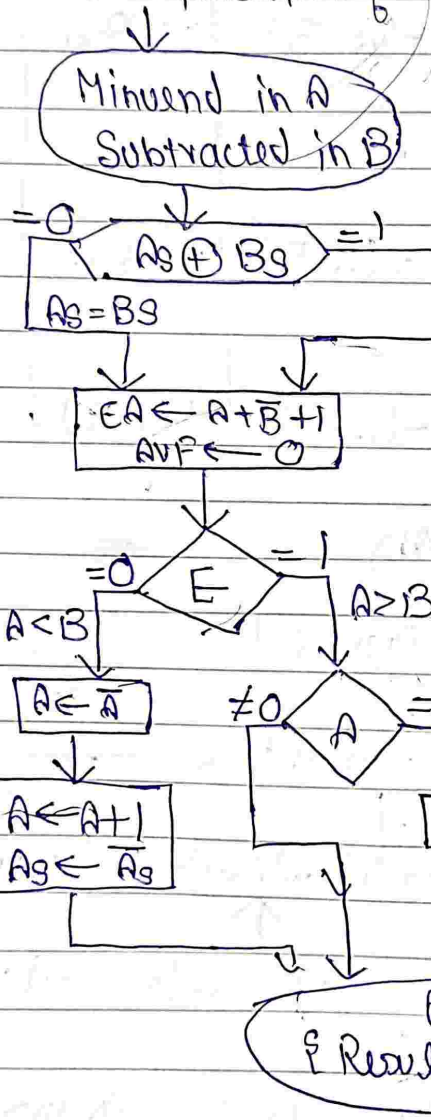
Hardware Implementation



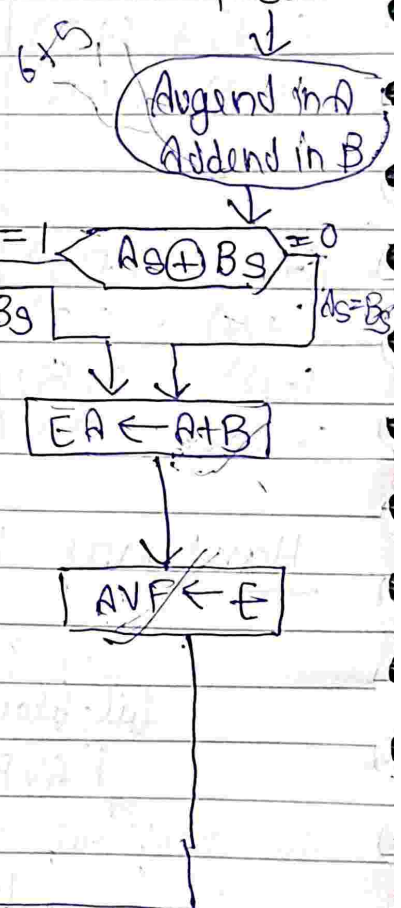
① When $m=0$, the output B is transferred to the adder, the input carry is 0, the adder equal to the sum

① When $m=1$, the 1's complement of B is applied to the adder, the input carry is 1, and output is $= A + \bar{B} + 1$ ^{2's} _{1's} $A - B$ (equal to $A - B$)

Subtract operation



Add operation



Ex

6-5

minuend (A) = 6
subtrahend (B) = 5

Binary = 0110
Binary = 0101

$(+6) - (+5)$
 $A_8 \oplus B_8 \rightarrow 0$

6-5 $\Rightarrow 1$

$\Rightarrow EA \leftarrow A + \bar{B} + 1$

$\Rightarrow \bar{B} + 1 \Rightarrow$ 2's complement

$\Rightarrow 0101$
1's $\Rightarrow 1010$
 $+1$

2's $\rightarrow 1011$

$\Rightarrow EA =$
$$\begin{array}{r} 0110 \\ + 1011 \\ \hline 10001 \end{array}$$

① 0001 A
 $\rightarrow$ extra carry

$\Rightarrow E=1, A > B, \neq 0$

A $\rightarrow 0001$

A₈ $\rightarrow 0$

Result = +1

$$\begin{array}{r} 0101 \\ 1010 \\ \hline \end{array}$$

Addition

6+5

$\Rightarrow$
$$\begin{array}{r} 0110 \\ 0101 \\ \hline 1011 \\ 222120 \end{array}$$

8+2+1

$\Rightarrow 11$
6+5=11

1011
0
 $\hline 1011$

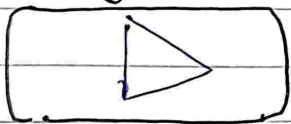
UNIT-02

LECTURE-02

COMPUTER ORGANIZATION & ARCHITECTURE

Topics :-

- ① MULTIPLICATION ✓
- ① Signed operand multiplication
- ① Booth algorithm ✓
- ① Array Multiplier



SUBSCRIBE = eMULTIATOMS

&
© MULTIATOMS PLUS

MULTIPLICATION ALGORITHM FOR SIGNED MAGNITUDE

- ① Multiplication of two fixed point binary numbers in signed magnitude representation is done with process of successive shift and add operation.
- ② The signed magnitude multiplication algorithm is a way of multiplying two numbers that are represent in signed magnitude form.

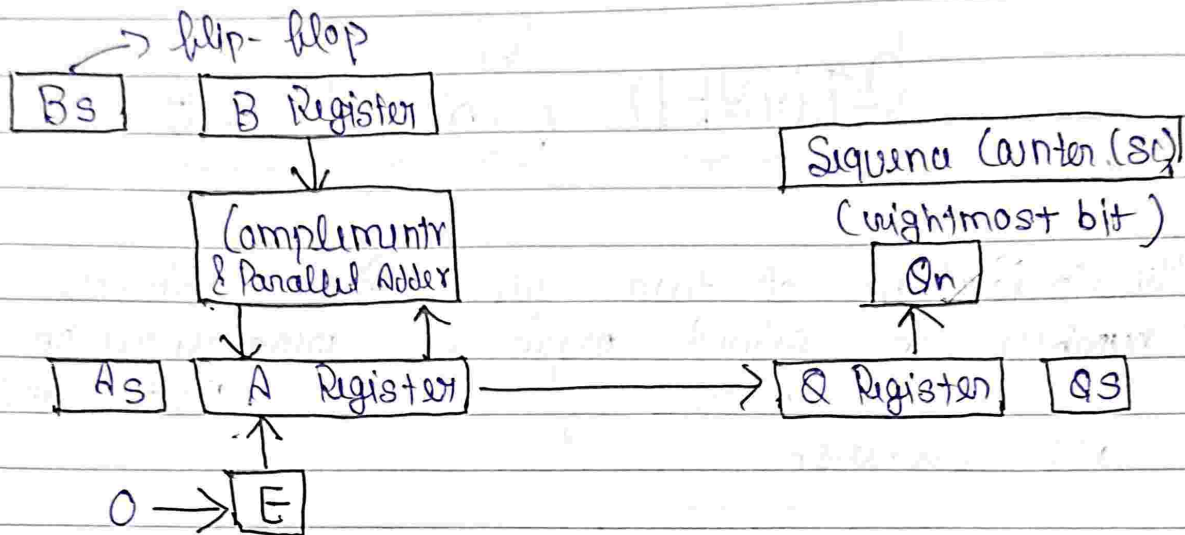
<u>Ex</u>	23	10111	Multiplicand
	19	x 10011	Multiplier
		10111	
		10111	
		00000	
		00000	
		10111	

437 110110101

$$\begin{array}{r}
 10111 \\
 \times 10011 \\
 \hline
 10111 \\
 00000 \\
 00000 \\
 10111 \\
 \hline
 110110101
 \end{array}$$

- ③ If the multiplier bit is 1, the multiplicand bit is copied down, otherwise (zeros) are copied down.

HARDWARE IMPLEMENTATION :-



$B \leftarrow$ Multiplicand, $Bs \leftarrow$ sign
 $Q \leftarrow$ Multiplier, $Qs \leftarrow$ sign.

① $SC \leftarrow$ no. of bits in multiplier.

② SC is decremented after performing each partial product

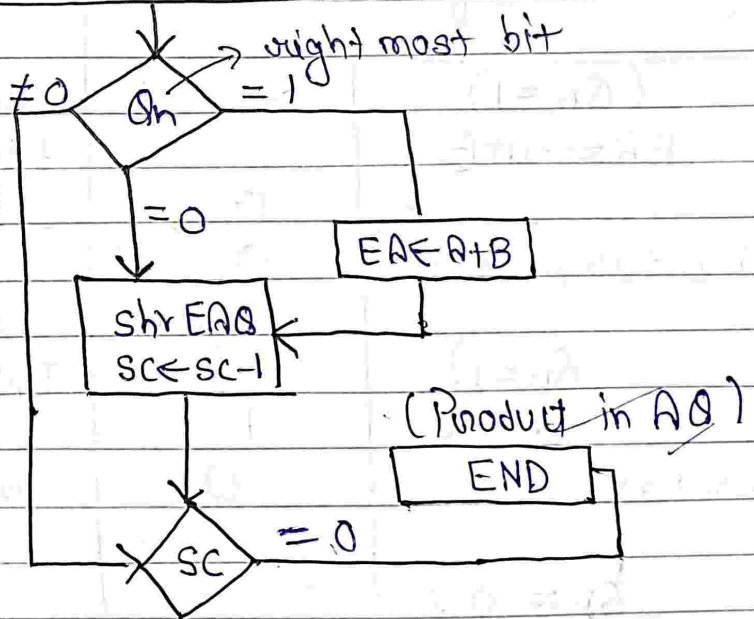
③ When SC is 0, processor halts and final product is formed

MULTIPLY OPERATION

Multiply Operation

Multiplicand in B
Multiplier in Q

$A_s \leftarrow Q_s \oplus B_s$ & $Q_s \leftarrow Q_s \oplus B_s$
 $A \leftarrow 0, E \leftarrow 0$
 $SC \leftarrow n-1$



Q Multiplicand = 23
 Multiplier = 19

Sol Multiplicand in B = 23
 Binary = 10111

Multiplier in Q = 19
 Binary = 10011

Operation	E	A	Q	SC
Initial conf.	0	00000	10011	5
($Q_n = 1$) $EA \leftarrow A+B$		00000 + 10111 ----- 10111		
sh _l EAQ, $SC \leftarrow SC-1$	0	01011	11001	4
($Q_n = 1$) $EA \leftarrow A+B$	1	01011 + 10111 ----- 00010		
sh _l EAQ, $SC \leftarrow SC-1$	0	10001	01100	3
($Q_n = 0$) sh _l EAQ, $SC \leftarrow SC-1$	0	01000	10110	2
($Q_n = 0$) sh _l EAQ, $SC \leftarrow SC-1$	0	00100	01011	1

$(Q_n = 1)$		00100		
$E \leftarrow A + B$		+ 10111		
	0	11011	01011	
$\text{Shr } EAB, SC \leftarrow SC-1$	0	$\rightarrow 01101$	$\rightarrow 10101$	0
Final Product in AB		0110110101		

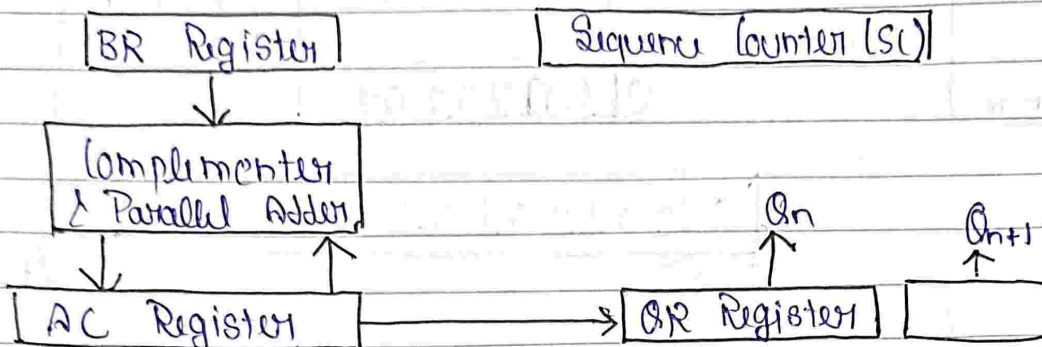
$$0110110101 = 437$$

A

Booth's algorithm

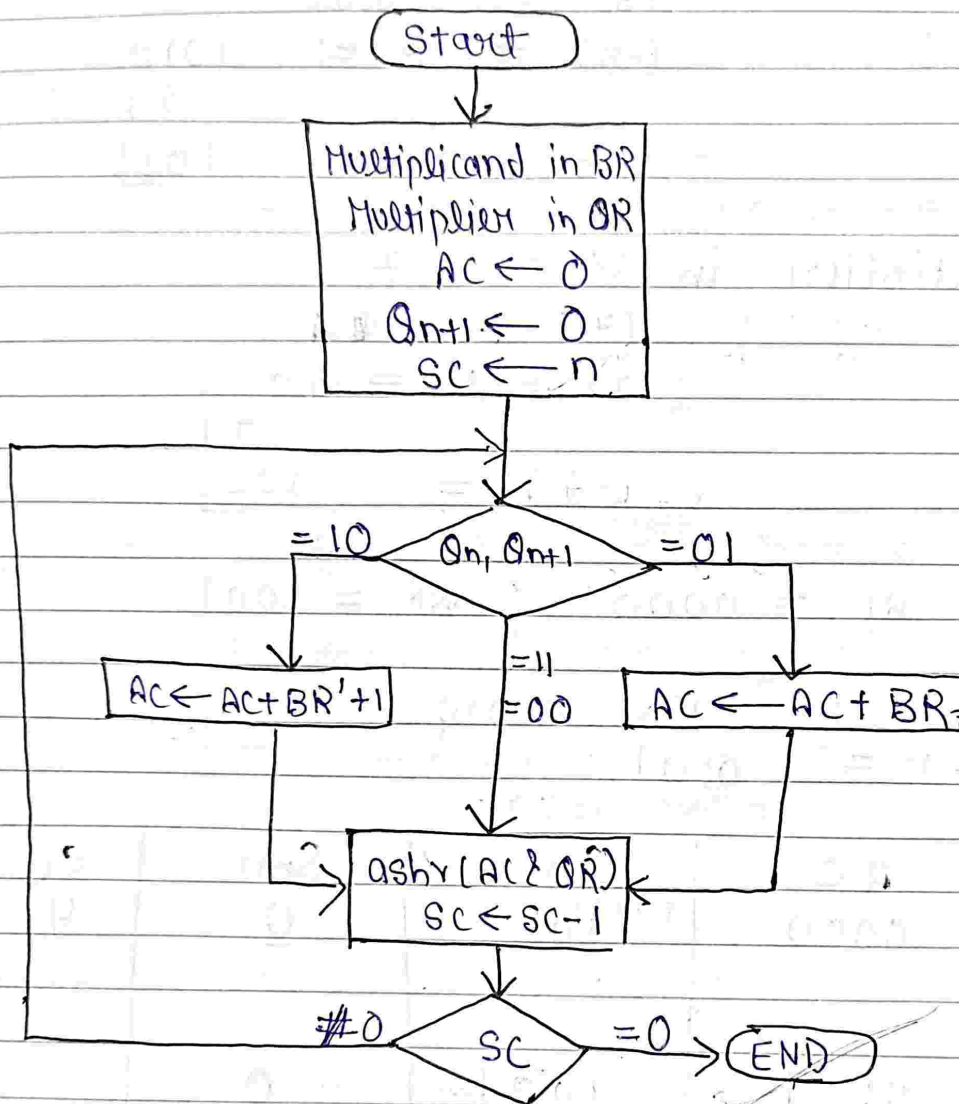
- ① Booth's algorithm is a special way to multiply two signed (positive or negative) binary numbers that are in 2's complement format
- ① It is also used to speed up the performance of the multiplication process
- ① It is very efficient algorithm
- ① It operates on the fact that:
 - $\rightarrow$ Strings of 0's in the multiplier: - Just shift to the right - no calculation needed
 - $\rightarrow$ If the bits are 1's: - Treat the group of 1's as one block, do one calculation to simplify things instead of repeating the same operation over and over

HARDWARE IMPLEMENTATION:



- ① Registers A, B and Q (also called AC, BR and QR): These are special storage areas in a computer that hold numbers we want to use for calculation like multiplication.
- ② Register Q (QR) holds the multiplier: - The register QR specifically stores the number we want to multiply.
- ③ Q_n designates the least significant bit: - The bit labeled Q_n is the rightmost bit in the QR register.
- ④ An extra flip-flop Q_{n+1} is appended to QR.
- ⑤ The extra flip-flop allow us to check the multiplier twice.

BOOTH ALGORITHM



Q Compute -5×-7 using the Booth's algorithm.

Sol Multiplicand in BR = -5

$$(5) \Rightarrow 0101$$

$$(-5) \Rightarrow \bar{5} \Rightarrow \begin{array}{r} 1010 \\ +1 \end{array}$$

$$(-5) \Rightarrow \underline{1011}$$

Multiplier in QR = -7

$$(7) = 0111$$

$$(-7) = \bar{7} = \begin{array}{r} 1000 \\ +1 \end{array}$$

$$(-7) \Rightarrow \underline{1001}$$

Initially AC = 0000, QR = 1001

$$BR = 1011, BR' = 0100$$

$$BR' + 1 = \begin{array}{r} 0101 \\ +1 \\ \hline 0110 \end{array}$$

Operation	AC	QR	Q _{n+1}	SC
Initial	0000	1001	0	4
AC + BR' + 1	0101	1001	0	
ashr	0010	1100	1	3
AC + BR	0010			
	+ 1011			
	<u>1101</u>	1100	1	
ashr	1110	1110	0	2

	1110	1110	0	
ashr	1111	0111	0	7
AC+BR'+1	1111 + 0101			
ashr	0100	0111	0	
Result	0010	0011	1	0
	00100011			= 35

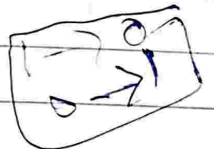
Q Show the systematic multiplication process of $(-15) \times (-16)$ using Booth's algorithm. [AITSU, 2022-23]

Multiplicand in BR $\Rightarrow (-15)$
 $(15) = 01111$
 $(-15) = 1^9 = 10000$
 $\quad \quad \quad + 1$
 $(-15) \Rightarrow \underline{10001}$

Multiplier in OR $= (-16)$
 $(16) = 10000$
 $(-16) = 1^9 = 01111$
 $\quad \quad \quad + 1$
 $(-16) \Rightarrow \underline{10000}$

initially AC = 00000, OR = 10000

BR = 10001, BR' = 01110
 $BR' + 1 = \underline{01111}$



Initial	AC	OR	Qn+1	sc
	00000	10000	0	5
ashr	00000	01000	0	4
ashr	00000	00100	0	3
ashr	00000	00010	0	2
ashr	00000	00001	0	1
AC + BR' + 1	01111	00001	0	
ashr	00111	10000	1	0

$$\text{Result} = 1001110000 = 240 \quad \text{Ans}$$

Q show the multiplication process using Booth algorithm, when the following binary numbers $(+13) \times (-15)$ are multiplied.
[AKTU 2021-2022]

Sol Multiplicand in BR = $(+13)$
 $(13) = 01101$

Multiplicand in BR = (-15)
 $(15) = 01111$
 $(-15) = 10000$
 $\quad \quad +1$
 $(-15) = 10001$

Initially AC = 00000, BR = 10001
 BR = 01101, BR' = 10010
 $BR' + 1 = 10011$

Initial	AC	BR	Q _{n+1}	SC
	00000	10001	0	5
AC + BR' + 1 ashv	10011 10001	10001 11000	0 1	4
AC + BR	11001 01101			
ashv	00110 00011	11000 01100	1 0	3

	00011	01100	0	
ashr	00001	10110	0	2
ashr	00000	11011	0	1
AC + BR' + 1 ashr	10011	11011	0	
	10001	11101	1	0
Result	11001	11101		

2's $\Rightarrow$ 0011000010

+1
0011000011 Result
 $\hookrightarrow$ -195

Q show the multiplication process using Both algorithm when the following numbers are multiplied $(-12) \times (-18)$.

Sol Answer = 0011011000 = 216

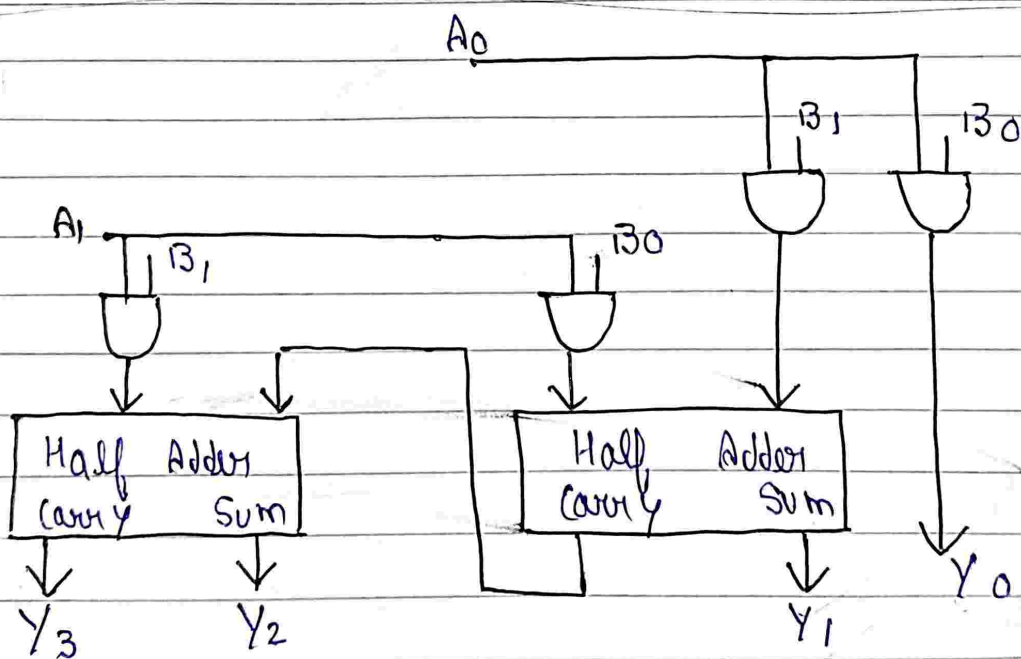
HOMEWORK QUESTION

Q Explain 2-bit by bit Array Multiplier. AKT 2021-2022

Sol $\odot$ An array multiplier is a digital combinational circuit used for multiplying two binary numbers by employing an array of full adders and half adders.

$\odot$ Multiplier circuit is based on add and shift algorithm.

$$\begin{array}{r}
 b_1 b_0 \\
 \times a_1 a_0 \\
 \hline
 a_0 b_1 a_0 b_0 \\
 a_1 b_1 a_1 b_0 \\
 \hline
 c_3 c_2 c_1 c_0
 \end{array}$$



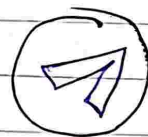
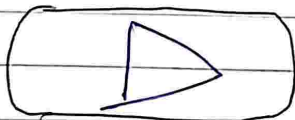
UNIT-01

LECTURE-03

COMPUTER ORGANIZATION & ARCHITECTURE

Topics:-

- ① Division Algorithm for Signed Magnitude
- ① Hardware Implementation
- ① Floating Point Representation
- ① Floating Point Arithmetic Operation
- ① Floating Point multiplication operation
- ① floating point division operation



SUBSCRIBE :

@MULTIATCH9

@MULTIATCH9+

DIVISION ALGORITHM FOR SIGNED MAGNITUDE

① The Division of two fixed-point binary number is the signed-magnitude representation is done by the cycle of successive compare, shift and subtract operation.

② The binary division is easier than the decimal division because the quotient digit is either 0 or 1.

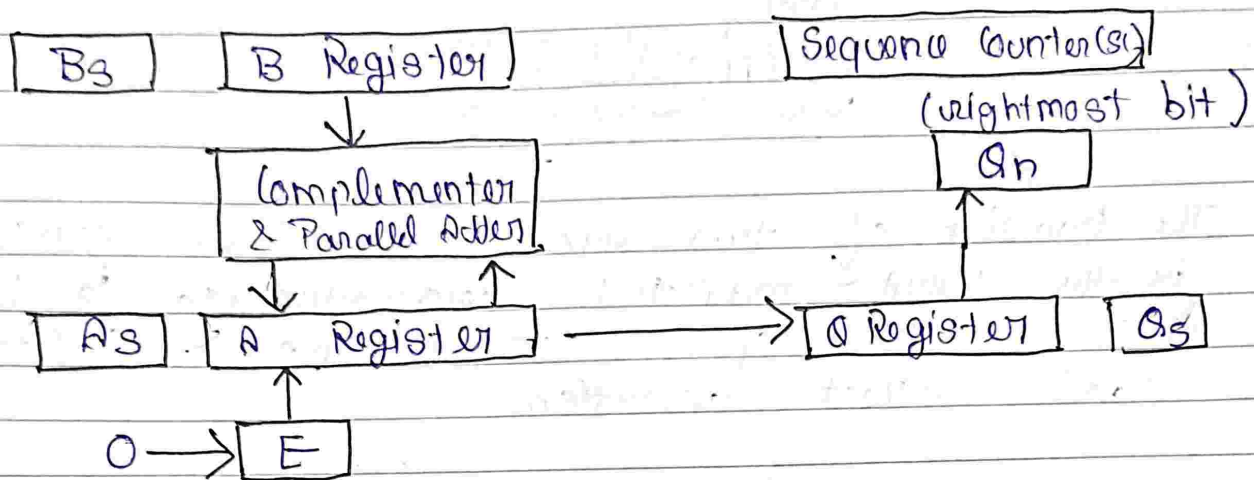
$$\begin{array}{r}
 10001 \overline{) 0111000000} \\
 \underline{010001} \\
 000110 \\
 \underline{-10001} \\
 0010100 \\
 \underline{10001} \\
 000110 \rightarrow \text{Remainder}
 \end{array}
 \quad
 \begin{array}{l}
 11010 \\
 \hline
 \text{Quotient}
 \end{array}$$

$\left. \begin{array}{l} \text{Dividend} < \text{Divisor, } R=0 \\ \text{Dividend} > \text{Divisor, } R=1 \end{array} \right\}$

Dividend (A): 0111000000 (448)
 Divisor (B): 10001 (17)

$$\begin{array}{r}
 17 \overline{) 448} \quad 26 \\
 \underline{34} \\
 108 \\
 \underline{102} \\
 6
 \end{array}$$

HARDWARE IMPLEMENTATION :-



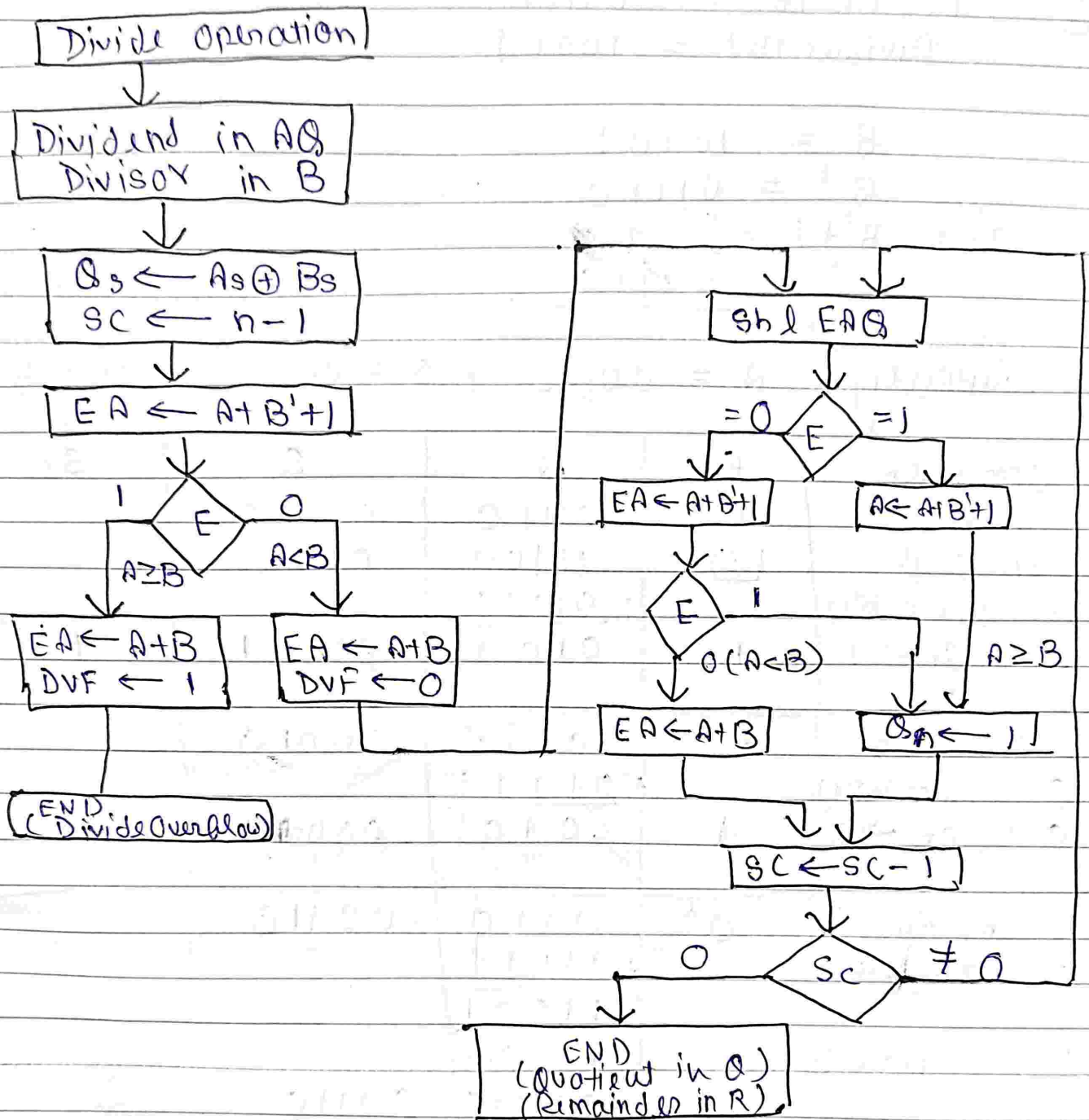
① This hardware implementation is identical to the multiplication.

② Here, Register B is used to store divisor and the double-length dividend is stored in Registers A and Q.

③ The information for the relative magnitude is given in E.

④ Sequence counter — no. of bits in the dividend

ALGORITHM FOR DIVIDE OPERATION



Q Dividend (A) = 01110
Divisor (B) = 10001

$$B = 10001$$

$$B' = 01110$$

$$2s = B' + 1 = \underline{\quad + 1 \quad}$$

$$\underline{01111}$$

initially A = 0000, Q = 00000, SC = 5

Operation	E	A	Q	SC
Initial	0	01110	00000	
shl EAQ	0	11100	00000	
E=0, EA ← A+B'		+ 01111		
E=1, Q _n → 1	1	01011	00001	4
shl EAQ	0	10110	00010	
E=0, EA ← A+B'		+ 01111		
E=1, Q _n → 1	1	00101	00011	3
shl EAQ	0	01010	00110	
E=0, EA ← A+B'		+ 01111		
		11001		
E ≠ 1, EA ← A-B		+ 10001		
	1	01010	00110	2
shl EAQ	0	10100	01100	
E=0, A+B'+1		+ 01111		
E=1, Q _n → 1	1	00011	01101	1

shl EAB	0	00110	11010
E=0, A+B'+1		+0111	
E≠1, A+B		10101	
		+10001	
	1	00110	11010

↓
↓
 Remainder Quotient

$$\left. \begin{array}{l} R = 00110 \\ Q = 11010 \end{array} \right\}$$

A

FLOATING POINT REPRESENTATION :-

- ① A floating point number usually has a decimal point. Ex 0, 3.14, 6.5.
- ① Various floating point representation are used in computer.
- ① In year 1985, IEEE 754 standard for floating point arithmetic established.
- ① IEC standard for floating-point arithmetic (IEEE 754) is a technical standard for floating point computation which was established by the Institute of Electrical Engineers (IEEE).

IEEE 754 has 3 basic components:-

Sign of Mantissa (s):-

0 represents a positive number while 1 represents a negative number

Biased / Modified exponent (E'):-

The exponent field needs to represent both positive and negative exponents.

A bias is added to the actual exponent in order to get the stored exponent

⇒ Normalized Mantissa (M):-

• The mantissa is part of a number in scientific notation or a floating-point number, consisting of its significant digits

• As we have only 2 digits i.e. 0 and 1. So a normalized mantissa is one with only one to the left of the decimal

IEEE 754 Floating Point can be:

Single Precision

32 bits

①

② Largest no: 2,147,483,647

③ Smallest no: -2,147,483,647

④ $E' = E + 127$

Double Precision

64 bits

9,223,372,036,854

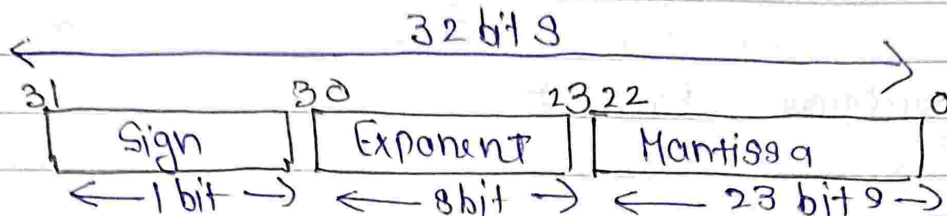
775,807

-9,223,372,036,854

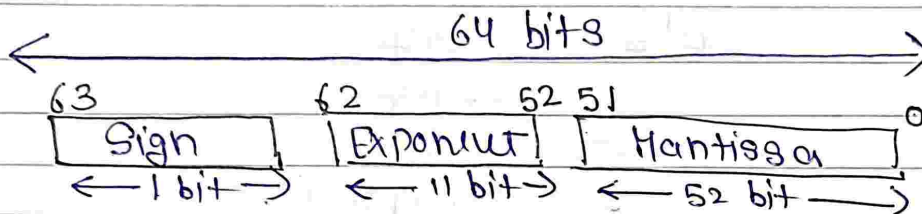
775,807

$E' = E + 1023$

Single Precision Floating Point Number:-



Double Precision Floating Point Number:-



Example (1460.125)₁₀

Sol ~~(1460)₁₀ = (10110110100)₂~~
~~(0.125)₁₀ = (0.001)₂~~

(1460)₁₀ = (10110110100)₂
 (0.125)₁₀ = (0.001)₂

(1460.125) = (10110110100.001)₂
 $\Rightarrow$ (10110110100.001)₂

$\rightarrow$ Normalise the number! -

Single Precision (1.N)₂ E-127
 Double Precision (1.N)₂ E-1023

$$(101101101001001)_2$$

$$\Rightarrow 1.0110110100001 \times 2^{10}$$

Single Precision format

$$S = 0$$

$$C = 10$$

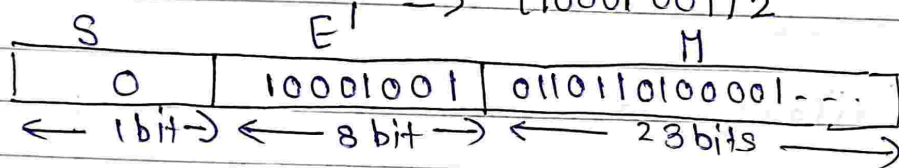
$$M = 0110110100001$$

$$E' = C + 127$$

$$\Rightarrow 10 + 127$$

$$\Rightarrow 137$$

$$\Rightarrow (10001001)_2$$



Double Precision format

$$S = 0$$

$$C = 10$$

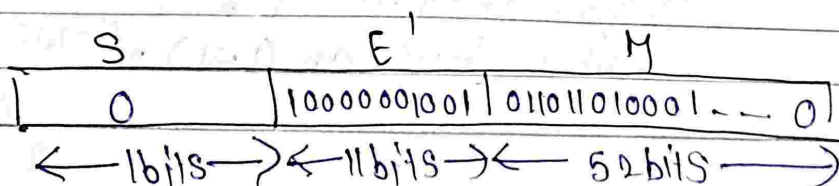
$$M = 0110110100001$$

$$E' = C + 1023$$

$$= 10 + 1023$$

$$\Rightarrow 1033$$

$$\Rightarrow (10000001001)_2$$



Q Explain IEEE-754 standard for floating point representation. Express $(314.175)_{10}$ in all two IEEE-754 models [AKTU 2022-23]

Sol $(314.175)_{10}$

$$(314)_{10} = (100111010)_2$$

$$(.175)_{10} = (0010110011 \dots)_2$$

$$(314.175)_{10} = (100111010.0010110011 \dots)_2$$

Normalise the number

$$\text{Single precision (1.N)}_2 \times 2^{E-127}$$

$$\text{double Precision (1.N)}_2 \times 2^{E-1023}$$

$$\Rightarrow (100111010.0010110011 \dots)_2$$

$$\Rightarrow 1.001110100010110011 \dots \times 2^8$$

Single Precision format

$$S = 0$$

$$C = 8$$

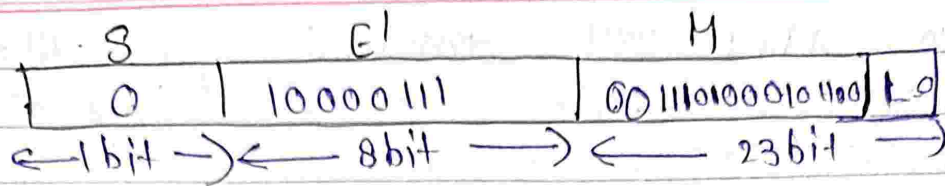
$$M = 001110100010110011 \dots$$

$$E' = C + 127$$

$$= 8 + 127$$

$$\Rightarrow 135$$

$$\Rightarrow (10000111)_2$$



Double Precision

$$S = 0$$

$$C = 8$$

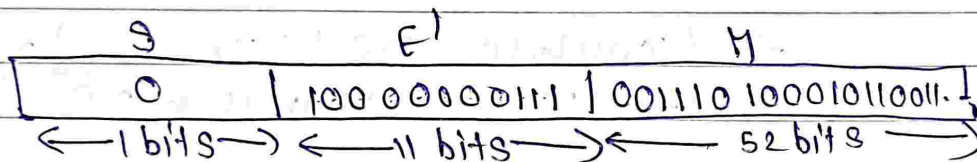
$$M = 001110100010110011...$$

$$E' = C + 1023$$

$$= 8 + 1023$$

$$\Rightarrow 1031$$

$$\Rightarrow (100000000111)_2$$



Q Represent the following decimal number in IEEE standard floating-point format in a single precision method (32 bit) representation method

(a) $(85.125)_{10}$

$$(85.125)_{10}$$

$$(85)_{10} = (1010101)_2$$

$$(0.125)_{10} = (.001)_2$$

$$(85.125)_{10} = (1010101.001)_2$$

Normalise the number:-

$$(1010101.001)_2$$

$$\Rightarrow 1.010101001 \times 2^6$$

$\Rightarrow$ Single Precision format

$$S = 0$$

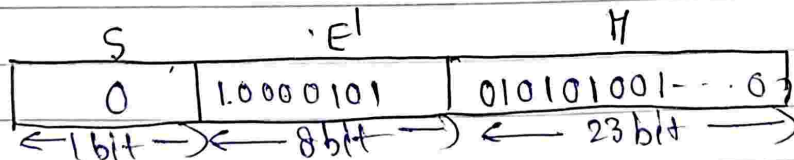
$$e = 6$$

$$M = 010101001$$

$$E = e + 127$$

$$\Rightarrow 6 + 127 = 133$$

$$\Rightarrow (10000101)_2$$



$$(b) \quad (-307.1875)_{10}$$

$$(307)_{10} = (100110011)_2$$

$$(0.1875)_{10} = (0.0011)_2$$

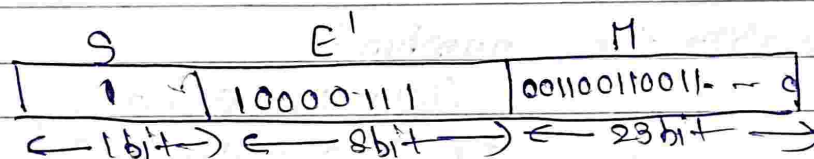
$$(-307.1875)_{10} = (100110011.0011)_2$$

Normalise the number

$$\Rightarrow (100110011.0011)_2$$

$$\Rightarrow 1.001100110011 \times 2^8$$

Single precision format



$$S = 1$$

$$e = 8$$

$$M = 001100110011$$

$$E' = e + 127$$

$$= 8 + 127$$

$$= 135$$

$$= (10000111)_2$$

Floating Point Arithmetic Operation

① Arithmetic operations on floating point numbers consist of addition, subtraction, multiplication and division.

② A floating point (FP) number is normalized if the Most significant Digit of the mantissa is non zero.

★ Floating-point addition and subtraction requires first an alignment of the radix point since the exponent part must be made equal.

★ Floating point Multiplication and division do not require an alignment of mantissas :-

- Product can be formed by multiplying two mantissas and adding the exponent.

- Division is accomplished by dividing the mantissas and subtracting the exponent.

B_s | B

b | BR

E | Parallel Adder

Parallel Adder &
Comparator

A_s | A_1 | A

a | AC

Q_s | Q

q | QR

Registers for floating Point Arithmetic Operation

- Three registers AC , BC and QR .
- Each register is subdivided into two parts
- Upper case letter — mantissa
- Lower case letter — exponent
- AC has a mantissa whose sign is in A_s and a magnitude that is in A
- The M.S.B of A is A_1 , The bit in this position must be a 1 to normalize the number
- BR is subdivided into B_s , B and b and Q into Q_s into Q and q
- A parallel-adder adds the two mantissas and loads the sum into A and the carry into E

Ex

Addition

$$\begin{array}{r} 0.534 \times 10^3 \\ + 0.712 \times 10^3 \\ \hline 1.246 \times 10^3 \end{array}$$

Register can not store overflow bit therefore perform shift right operation

$$0.124 \times 10^4$$

Subtraction

$$\begin{array}{r} 0.534 \times 10^3 \\ - 0.521 \times 10^3 \\ \hline 0.013 \times 10^3 \end{array}$$

If MSB is zero. It is known as underflow condition and to overcome this perform shift left operation

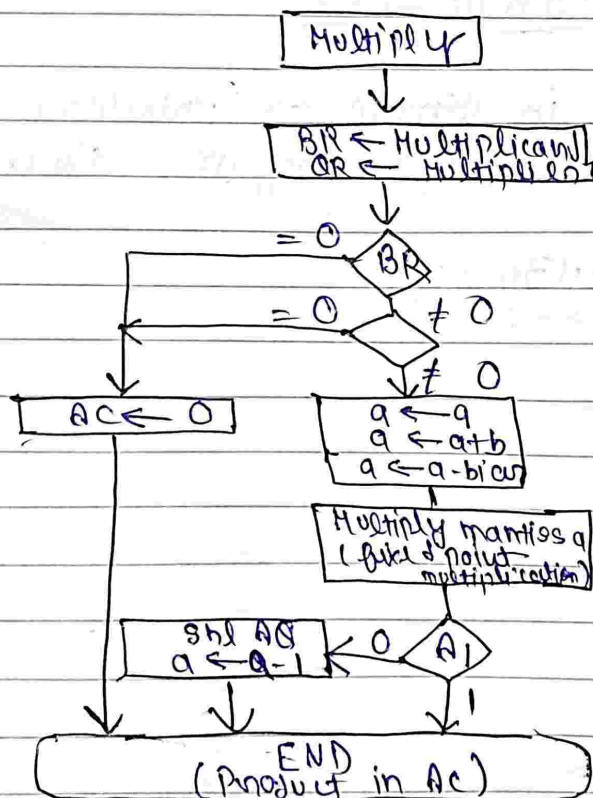
$$0.130 \times 10^2$$

Floating Point Multiplication

Floating point multiplication does not require an alignment of mantissas, product can be formed by multiplying two mantissas and adding the exponents

The algorithm can be divided into four steps

- ★ check for zeroes
- ★ Add the exponents
- ★ Multiply the Mantissas
- ★ Normalize the result



$$\begin{array}{rcl} \textcircled{0} & 0.12 \times 10^4 & \longrightarrow \text{BR} \\ \times & 0.23 \times 10^3 & \longrightarrow \text{QR} \\ \hline & 0.0276 \times 10^7 & \end{array}$$

$$\text{BR} = 0, \quad \text{AC} = 0$$

$$\text{QR} = 0$$

$$A_1 = 0, \quad \text{shl } A_0$$

$$\boxed{00.276 \times 10^6}$$

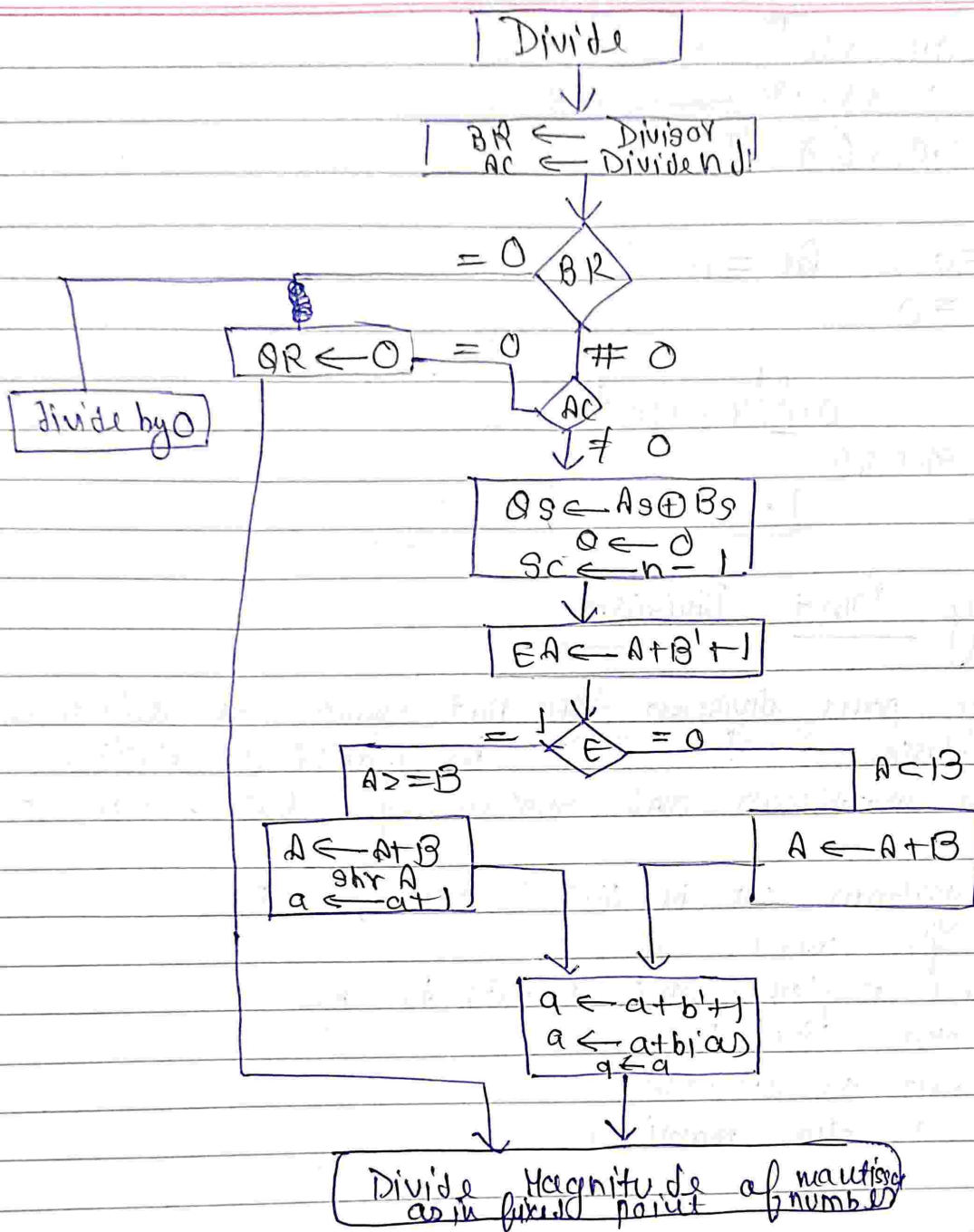
A₁

Floating Point Division

① floating point division does not require an alignment of mantissas, division can be formed by dividing two mantissas and subtracting the exponent

① The algorithm can be divided into five steps:-

- check for zeroes
- initialize registers and evaluate the sign
- Align the dividend
- Subtract the exponent
- Divide the mantissas



THANK YOU FOR WATCHING

LIKE SHARE SUBSCRIBE

SUBSCRIBE . @MULTIATOMS

AND

@MULTIATOMS +